

TP 3 : Modèle de Poisson et mélanges

Modèles probabilistes pour l'apprentissage - MPA

Hiba HANINI, Marwane HAJJY, Hiba FAHLI

24/10/2025

Première partie : Modèle de Poisson simple

Q1 . Déterminer la loi générative des données, y , dans ce modèle.

Notons : $y = (y_1, y_2, \dots, y_n)$. On a $\forall i \in \{1, \dots, n\}$, $y_i \sim P(\theta)$ et sont i.i.d. Notons $s_n = \sum_{i=1}^n y_i$.
On a :

$$\begin{aligned} p(y \mid \theta) &= \prod_{i=1}^n p(y_i \mid \theta) \\ &= \prod_{i=1}^n \frac{\theta^{y_i} e^{-\theta}}{y_i!} \end{aligned}$$

Donc :

$$p(y \mid \theta) = \frac{\theta^{s_n} e^{-n\theta}}{\prod_{i=1}^n y_i!}$$

Q2. Déterminer la loi a posteriori du paramètre θ . Quelle est l'espérance de la loi a posteriori

On a d'après Bayes :

$$\begin{aligned} p(\theta \mid y) &\propto p(y \mid \theta) p(\theta) \\ &\propto \frac{\theta^{s_n} e^{-n\theta}}{\prod_{i=1}^n y_i!} \frac{1}{\theta} \\ &\propto \theta^{s_n-1} e^{-n\theta} \end{aligned}$$

Ainsi, on reconnaît le noyau d'une loi Gamma :

$$\theta \mid y \sim \Gamma(s_n, n)$$

Et par suite :

$$\mathbb{E}(\theta \mid y) = \frac{s_n}{n}$$

Q3. Rappeler le principe de l'algorithme de Metropolis-Hasting. Ecrire dans un programme en R une version de cet algorithme.

Principe : Pour simuler une loi cible $\pi(\theta)$, on construit une chaîne de Markov. À l'étape t , on propose un candidat θ^* selon une loi instrumentale $q(\cdot|\theta^{(t)})$. On accepte ce candidat avec une probabilité $\alpha = \min(1, \frac{\pi(\theta^*)q(\theta^{(t)}|\theta^*)}{\pi(\theta^{(t)})q(\theta^*|\theta^{(t)})})$.

Ici, nous utilisons une loi instrumentale Exponentielle fixe $q(\theta) \sim \mathcal{E}(\lambda_{inst})$.

```
set.seed(123)

algorithm_metropolis <- function(y, theta.0, nit) {
  theta.post <- numeric(nit+1)
  theta.post[1] <- theta.0
  n <- length(y)
  sn <- sum(y)

  lambda_inst <- 1/mean(y)

  for (i in 1:nit) {
    theta.old <- theta.post[i]

    theta.prop <- rexp(1, rate = lambda_inst)

    log_pi_prop <- (sn - 1) * log(theta.prop) - n * theta.prop
    log_pi_old <- (sn - 1) * log(theta.old) - n * theta.old

    log_q_prop <- dexp(theta.prop, rate = lambda_inst, log = TRUE)
    log_q_old <- dexp(theta.old, rate = lambda_inst, log = TRUE)

    log_ratio <- (log_pi_prop - log_pi_old) + (log_q_old - log_q_prop)

    if (log(runif(1)) < log_ratio) {
      theta.post[i+1] <- theta.prop
    } else {
      theta.post[i+1] <- theta.old
    }
  }
  return (theta.post)
}
```

Q4. Fournir une estimation ponctuelle du paramètre theta.

```
lambda0 <- 5
n_sim <- 200
y_sim <- rpois(n_sim, lambda0)

# Simulation
theta.post <- algorithm_metropolis(y_sim, theta.0 = 1, nit = 10000)

# Burn-in (suppression du début de chaîne)
theta.post <- theta.post[-(1:500)]

cat("Vraie valeur du paramètre :", lambda0, "\n")

## Vraie valeur du paramètre : 5

cat("Moyenne a posteriori :", mean(theta.post), "\n")
```

```
## Moyenne a posteriori : 5.062624
cat("Médiane a posteriori :", median(theta.post), "\n")

## Médiane a posteriori : 5.060853
print(quantile(theta.post, c(0.025, 0.975)))

##      2.5%      97.5%
## 4.782504 5.399652
```

Q5. Représenter graphiquement l'histogramme de la loi a posteriori.

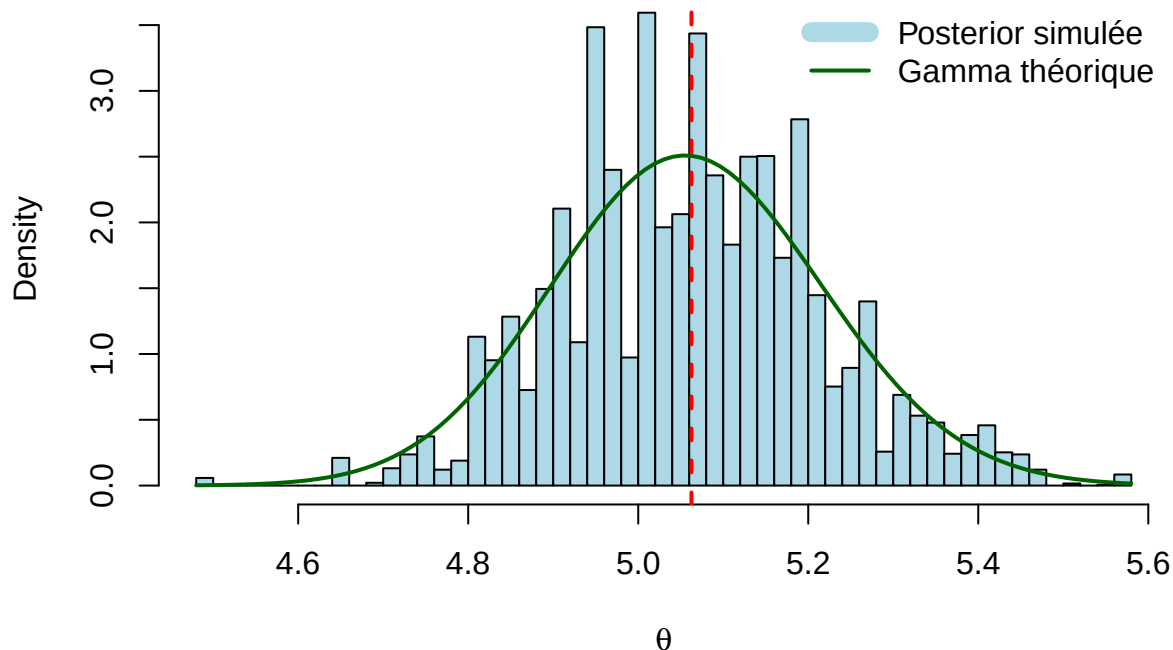
```
hist(theta.post, probability = TRUE,
      main = "Posterior simulée vs. Gamma théorique",
      xlab = expression(theta), col = "lightblue", breaks = 40)

# Superposition de la loi théorique Gamma(sn, n)
curve(dgamma(x, shape=sum(y_sim), rate=n_sim),
      col="darkgreen", lwd=2, add=TRUE)

legend("topright",
      legend = c("Posterior simulée", "Gamma théorique"),
      col = c("lightblue", "darkgreen"),
      lwd = c(10, 2), bty = "n")

abline(v = mean(theta.post), col = "red", lwd = 2, lty = 2)
```

Posterior simulée vs. Gamma théorique



Q6. Déterminer la loi prédictive a posteriori d'une nouvelle donnée $\tilde{y}$. Critiques.

Calcul théorique : La loi prédictive est donnée par :

$$p(\tilde{y} = k \mid y) = \int_0^{\infty} p(\tilde{y} = k \mid \theta) p(\theta \mid y) d\theta$$

Avec $\tilde{y} \mid \theta \sim \mathcal{P}(\theta)$ et $\theta \mid y \sim \Gamma(s_n, n)$, on obtient :

$$p(\tilde{y} = k \mid y) = \int_0^\infty \frac{\theta^k e^{-\theta}}{k!} \cdot \frac{n^{s_n}}{\Gamma(s_n)} \theta^{s_n-1} e^{-n\theta} d\theta$$

Après calcul (intégrale Gamma), on identifie une **Loi Binomiale Négative** :

$$p(\tilde{y} = k \mid y) = \binom{k + s_n - 1}{k} \left(\frac{n}{n+1} \right)^{s_n} \left(\frac{1}{n+1} \right)^k$$

Simulation et Application aux données “Chamilo” :

```
y_chamilo <- scan("TP3_MPA_2025.txt", quiet=TRUE)

theta_chamilo <- algorithme_metropolis(y_chamilo, theta.0 = 0.5, nit = 5000)
theta_clean <- theta_chamilo[-(1:500)] # Burn-in

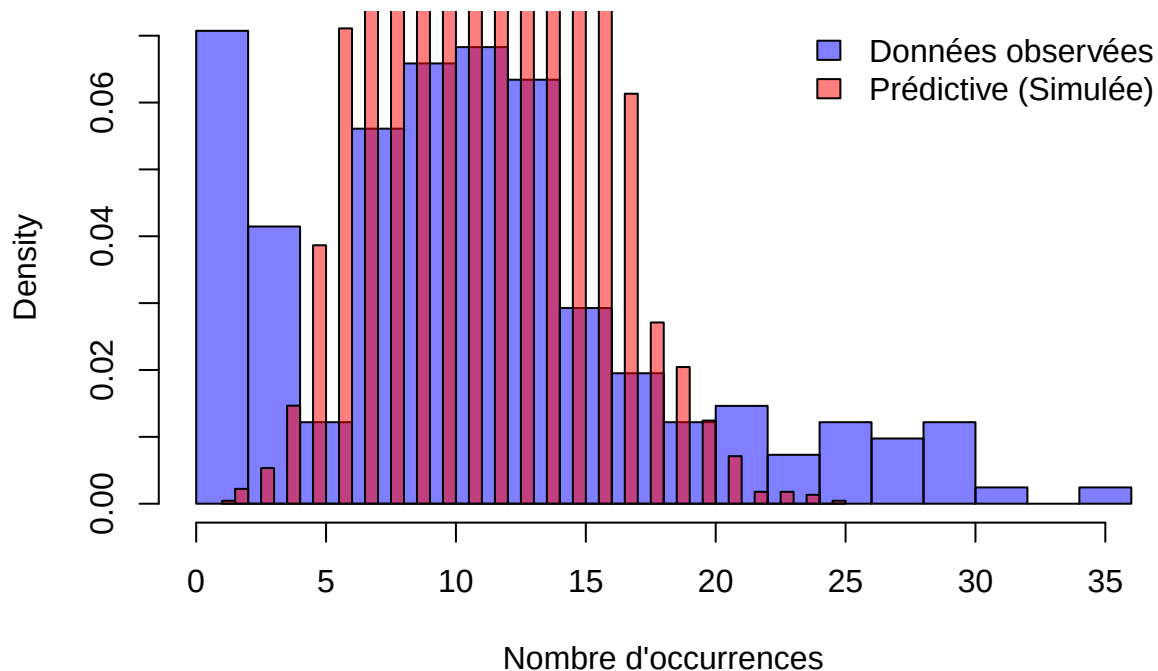
y_pred <- rpois(length(theta_clean), lambda = theta_clean)

hist(y_chamilo, probability = TRUE, col = rgb(0, 0, 1, 0.5),
     main = "Données observées vs Loi Prédictive",
     xlab = "Nombre d'occurrences", breaks = 20)

hist(y_pred, probability = TRUE, col = rgb(1, 0, 0, 0.5),
     add = TRUE, breaks = 40)

legend("topright", legend = c("Données observées", "Prédictive (Simulée)"),
     fill = c(rgb(0, 0, 1, 0.5), rgb(1, 0, 0, 0.5)), bty = "n")
```

Données observées vs Loi Prédictive



```
cat("Moyenne empirique :", mean(y_chamilo), "\n")
```

```
## Moyenne empirique : 11.24878
```

```
cat("Variance empirique :", var(y_chamilo), "\n")
```

```
## Variance empirique : 56.48192
```

Critique : On observe une forte **sur-dispersion** (Variance » Moyenne), ce qui contredit l'hypothèse de Poisson ($\mathbb{E} = \mathbb{V}$). énorme barre bleue à $y = 0$. La simulation (barres roses) ne prédit quasiment aucun zéro (une moyenne estimée autour de 11, la probabilité qu'une loi de Poisson génère un 0 est (e^{-10})). Le modèle considère les zéros observés comme des événements impossibles. Les données bleues semblent avoir deux bosses : un pic à 0 et un second nuage centré autour de 10-12. Le modèle proposé est unimodal (une seule bosse rose). Il cherche un compromis unique et "moyenne" tout le monde, ratant ainsi la structure à deux régimes.

Seconde partie : Mélanges de Poisson

Q1. Décrire la loi $p(y|z, \theta)$ soit $I_k = \{i : z_i = k\}$.

$$p(y|z, \theta) = \prod_{k=1}^K \prod_{i \in I_k} p(y_i | \theta_k) = \prod_{k=1}^K \prod_{i \in I_k} \frac{\theta_k^{y_i} e^{-\theta_k}}{y_i!}$$

Q2. Décrire la loi a posteriori du vecteur (z, θ) .

D'après Bayes, $p(z, \theta|y) \propto p(y|z, \theta)p(z)p(\theta)$. Avec $p(z) \propto 1$ et $p(\theta_k) \propto 1/\theta_k$:

$$p(z, \theta|y) \propto \left(\prod_{k=1}^K \prod_{i \in I_k} \frac{\theta_k^{y_i} e^{-\theta_k}}{y_i!} \right) \times \left(\prod_{k=1}^K \frac{1}{\theta_k} \right)$$

Q3. Calculer la probabilité conditionnelle $p(z_i = k|y, \theta)$.

Pour une donnée i , la probabilité qu'elle appartienne au groupe k ne dépend que de sa vraisemblance dans ce groupe (le prior sur z étant uniforme).

$$p(z_i = k|y_i, \theta) = \frac{p(y_i | \theta_k)}{\sum_{j=1}^K p(y_i | \theta_j)} = \frac{\theta_k^{y_i} e^{-\theta_k}}{\sum_{j=1}^K \theta_j^{y_i} e^{-\theta_j}}$$

Q4. Montrer que la loi conditionnelle de θ_k est une Gamma.

On regarde la loi jointe en ne gardant que les termes dépendant de θ_k :

$$\begin{aligned} p(\theta_k | \theta_{-k}, y, z) &\propto \frac{1}{\theta_k} \prod_{i \in I_k} (\theta_k^{y_i} e^{-\theta_k}) \\ &\propto \theta_k^{-1} \cdot \theta_k^{\sum_{i \in I_k} y_i} \cdot e^{-n_k \theta_k} \\ &\propto \theta_k^{n_k \bar{y}_k - 1} \exp(-n_k \theta_k) \end{aligned}$$

C'est le noyau d'une loi **Gamma** de paramètres de forme $\alpha = n_k \bar{y}_k$ (somme des y du groupe) et de taux $\beta = n_k$ (taille du groupe).

Q5. Implantation de l'algorithme d'échantillonnage de Gibbs.

```

gibbs_melange <- function(y, K, nit) {
  n <- length(y)

  # Initialisation aléatoire
  z <- sample(1:K, n, replace = TRUE)
  theta <- rep(NA, K)

  # Initialisation des theta intelligemment pour éviter les bugs (moyenne des groupes)
  for(k in 1:K) {
    if(sum(z==k)>0) theta[k] <- mean(y[z==k]) else theta[k] <- mean(y)
  }

  theta_history <- matrix(NA, nrow = nit, ncol = K)

  for (it in 1:nit) {

    # --- Etape 1 : Simulation des z (Allocation) ---
    for (i in 1:n) {
      # Calcul des log-vraisemblances pour stabilité numérique
      log_probs <- y[i] * log(theta) - theta
      # Log-Sum-Exp trick
      probs <- exp(log_probs - max(log_probs))
      probs <- probs / sum(probs)
      z[i] <- sample(1:K, 1, prob = probs)
    }

    # --- Etape 2 : Simulation des theta ---
    for (k in 1:K) {
      yk_sub <- y[z == k]
      nk <- length(yk_sub)
      syk <- sum(yk_sub)

      # Attention au cas (rare) où nk=0 ou syk=0
      # La loi Gamma(0,0) n'est pas définie (prior impropre).
      # Ici on suppose qu'il y a toujours des données, sinon on garde l'ancien theta
      if (nk > 0 && syk > 0) {
        theta[k] <- rgamma(1, shape = syk, rate = nk)
      }
    }
    theta_history[it, ] <- theta
  }

  return(list(theta_hist = theta_history, z_final = z))
}

```

Q6. Estimation de l'espérance et des proportions.

- **Espérance de θ_k** : On calcule la moyenne empirique des chaînes de Markov générées `theta_history` (après avoir retiré la période de chauffe).
- **Proportions** : On estime la proportion du groupe k par $\hat{\pi}_k = \frac{\text{Nombre de } z_i=k}{n}$.

Q7. Application aux données réelles et choix de K .

On teste le modèle pour $K = 2$ et $K = 3$.

```

# Lancement du Gibbs
set.seed(123)
res_K2 <- gibbs_melange(y_chamilo, K=2, nit=2000)
res_K3 <- gibbs_melange(y_chamilo, K=3, nit=2000)

# Burn-in
burn <- 500
est_theta_K2 <- colMeans(res_K2$theta_hist[-(1:burn), ])
est_theta_K3 <- colMeans(res_K3$theta_hist[-(1:burn), ])

# Proportions finales (basées sur la dernière itération)
props_K2 <- table(factor(res_K2$z_final, levels=1:2))/length(y_chamilo)
props_K3 <- table(factor(res_K3$z_final, levels=1:3))/length(y_chamilo)

# Fonction densité mélange
d_melange <- function(x, thetas, props) {
  d <- 0
  for(k in 1:length(thetas)) d <- d + props[k]*dpois(x, thetas[k])
  return(d)
}

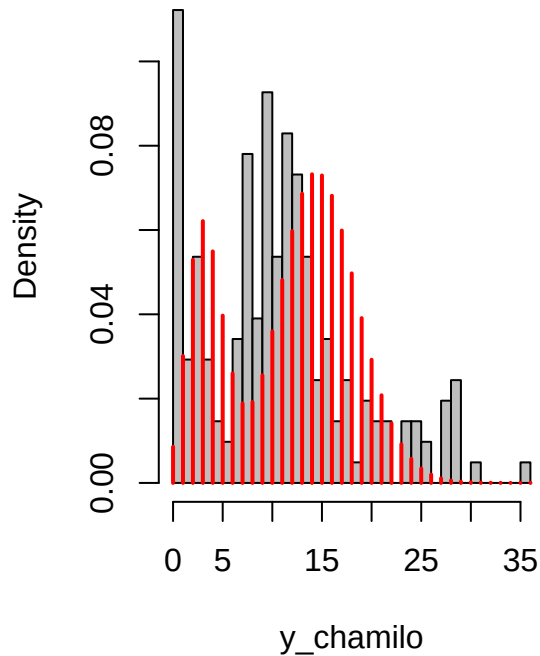
# Affichage
par(mfrow=c(1,2)) # Deux graphiques côte à côte

# K=2
hist(y_chamilo, freq=FALSE, breaks=30, main="Mélange K=2", col="grey")
lines(0:max(y_chamilo), d_melange(0:max(y_chamilo), est_theta_K2, props_K2),
      col="red", lwd=2, type="h")

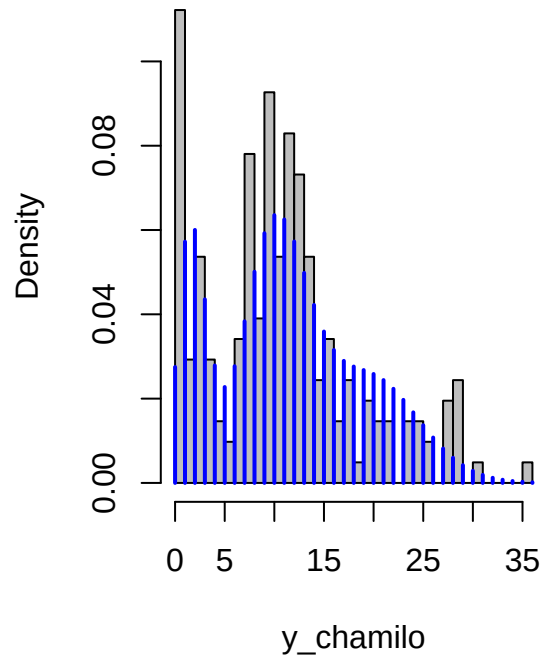
# K=3
hist(y_chamilo, freq=FALSE, breaks=30, main="Mélange K=3", col="grey")
lines(0:max(y_chamilo), d_melange(0:max(y_chamilo), est_theta_K3, props_K3),
      col="blue", lwd=2, type="h")

```

Mélange K=2



Mélange K=3



```
cat("Theta estimés (K=2) :", est_theta_K2, "\n")
```

```
## Theta estimés (K=2) : 14.94367 3.514139
```

```
cat("Theta estimés (K=3) :", est_theta_K3, "\n")
```

```
## Theta estimés (K=3) : 20.48301 2.077845 10.63095
```

Critique finale et Choix de K : L'analyse visuelle des histogrammes permet de voir si $K = 2$ suffit à capturer les différents modes (bosses) des données. Si l'ajout d'un troisième groupe ($K = 3$) superpose deux moyennes très proches ou laisse un groupe vide, alors $K = 2$ est préférable (principe de parcimonie). Le modèle de mélange corrige le défaut du modèle simple : il permet une variance globale plus grande que la moyenne (sur-dispersion) grâce à l'hétérogénéité des composantes.